Noesis: Selective Memory Injection for Cross-Tool AI Conversation Consistency

Peizheng Du
Independent Researcher
peterdu0628@gmail.com

October 2026

Abstract

Users converse with multiple AI assistants daily, yet each tool's memory is isolated: a stance expressed in one tool is unknown to the others, and each conversation starts from scratch. Shared and evolving memory layers already exist; this paper studies a narrower design choice: an explicit admission rule that decides whether a captured statement is firm enough to enter the prompt at all. I present **Noesis**, a local-first memory layer that (1) extracts typed *thoughts* from conversations across tools into a shared per-user store, (2) gates injection behind a confidence lifecycle in which thoughts mature from *tentative* to *settled* based on repetition, assertion strength, cross-tool corroboration, and contradiction signals, and (3) caps injected context with a token budget. Across three models (Gemini Flash-Lite, gemma3:4b, qwen2.5:3b) on 110 paired questions each, memory injection raises the keyword-hit rate on synthetic personal questions from 41.8% to 85.5%, 34.5% to 82.7%, and 37.3% to 62.7% respectively (+43.6, +48.2, and +25.5 percentage points; all $p < 0.001$, McNemar's test). At 500 stored memories, Noesis keeps 9 of 10 targets at rank 1 with about 207 injected tokens, 35 times fewer than injecting every memory (MRR 0.90 vs. 0.002); a standard top-5 RAG ranks as well with fewer tokens, so Noesis's distinct gain is filtering: where ungated retrieval surfaced tentative or hedged statements 19 and 13 times in two constructed tests, Noesis surfaced none; the gate filters by how firmly something was said, not by whether it is true. On strictly cross-tool queries, Noesis retrieves 37% where per-tool isolation is structurally bound to 0%. I release the system, the experiment harness, and the saved raw results.

Keywords: large language models; LLM memory; retrieval-augmented generation; personalization; cross-tool memory; confidence gating; local-first software

1 Introduction

Heavy users of AI assistants switch between ChatGPT, Claude, Gemini, and local models throughout the day. Each tool maintains its own isolated memory: a database preference carefully explained to Claude must be re-explained to GPT; a design decision reached with Gemini is invisible to Claude. The reasoning a user performs across tools, such as their positions, preferences, and open questions, evaporates at every tool boundary.

Two families of systems address parts of this problem. *Memory layers* such as Mem0 [3] extract and consolidate facts from conversations; OpenMemory MCP [18] shares one local store across MCP clients; and Zep [22] and Hindsight [13] track how facts change and how firmly opinions are held (§2). *Retrieval-augmented generation* (RAG) [14] retrieves relevant context for each query, but naive variants inject everything retrieved; as personal memory grows into the hundreds of items,

injected context grows linearly, burying the relevant signal and inflating cost (I measure 7,192 tokens at n=500).

Noesis is built on a different premise: *what* gets injected matters as much as *whether* anything is injected. Thoughts should earn their way into the context the way beliefs earn their way into a mind: through repetition, consistency, and survival against contradiction. Earning here means passing an admission rule, not being verified as true.

This paper makes the following contributions:

1. **A confidence lifecycle for conversational memory** (§3). Thoughts are typed (identity, preference, position, question, synthesis, event, contradiction) and mature tentative→provisional→settled via a four-signal scorer, with type-specific time decay and cluster-aware supersession that reduced stale-stance leakage from 93% to 7% (§3.3).

2. **Gated injection that scales** (§5.2–§5.3). At 500 memories Noesis injects ∼207 tokens (35 times fewer than injecting everything) while holding MRR at 0.90; a standard top-5 RAG matches that ranking at lower cost, and Noesis's gate is what keeps tentative and hedged statements out (0 vs. 19 and 0 vs. 13 in constructed tests).

3. **Quantified cross-tool memory sharing** (§5.4). With a shared per-user store, Noesis retrieves 37% of strictly cross-tool queries versus 0% for per-tool isolation (50% vs. 7.5% over all 40 scenarios)—a gap that is architecturally impossible for isolated systems to close.

4. **An A/B result across three model tiers** (§5.1). On identical n=110 question sets, memory injection significantly raises the keyword-hit rate for a cloud model (Gemini Flash-Lite, +43.6pp), a mid-tier local model (gemma3:4b, +48.2pp), and a small local model (qwen2.5:3b, +25.5pp); all $p < 0.001$.

The system is local-first: embeddings and retrieval run on-device (sqlite-vec + all-MiniLM-L6-v2), and conversation text leaves the machine only for optional LLM-based extraction or when the proxy forwards a request, with its injected memories, to a cloud model. All code, the experiment harness, the saved raw results, and a verification script that recomputes every reported number from raw JSON are released.[1]

2 Related Work

Sources were checked on 17 October 2026; the references mark preprints and vendor announcements, and `docs/related_systems.md` tabulates what each cited version describes.

Memory for LLM agents. MemGPT [20] treats the LLM as an operating system paging memory between contexts; Letta extends this with archival storage. Mem0 [3] extracts and consolidates facts with an LLM pipeline, reporting gains over full-history baselines, and has a graph-based variant; A-MEM [26] links memories into an evolving Zettelkasten-style network, and Zep [22] stores them in a temporal knowledge graph. Zep invalidates contradicted facts with an LLM, more generally than Noesis's supersession rules (§3.3), but describes no threshold that keeps a valid fact out. Generative Agents [21] rank memories by recency, importance, and relevance, and MemoryBank [30] lets them fade along an Ebbinghaus forgetting curve, the closest precedents for Noesis's type-specific decay. OpenMemory MCP [18] already shares one local store across MCP clients

[1] https://github.com/AngryKiwiWithLegs/Noesis

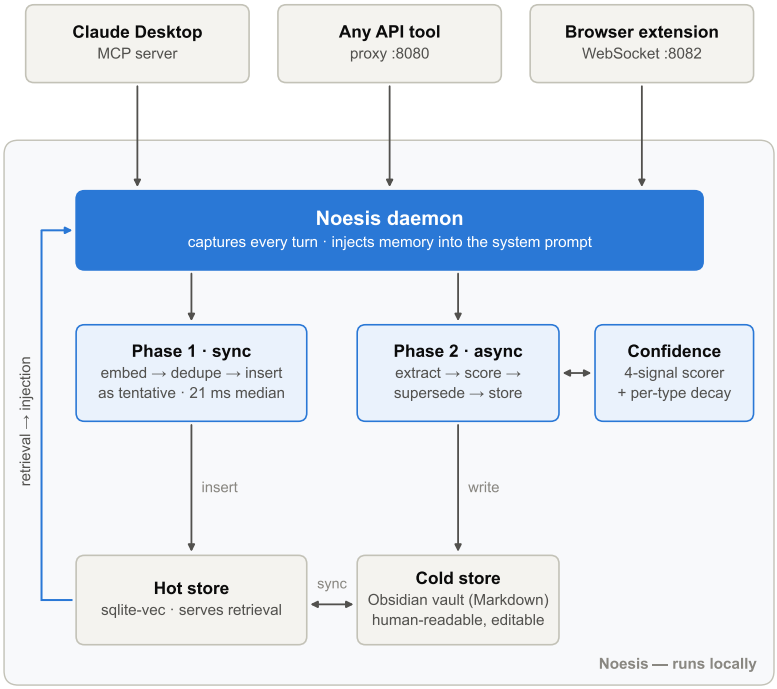


Figure 1: Noesis system architecture. Tools connect through an MCP server, an OpenAI-compatible proxy, or a WebSocket; Phase 1 stores the utterance synchronously (median 21 ms) and Phase 2 asynchronously extracts, scores, and links the thought. Hot store (sqlite-vec) serves retrieval; cold store (Obsidian vault) keeps memory human-readable and editable.

such as Cursor and Claude Desktop. Noesis differs in (a) an explicit admission rule, under which items below provisional status are never injected and which the OpenMemory announcement does not describe, and (b) its entry points beyond MCP: an OpenAI-compatible proxy and a browser extension make clients outside MCP first-class participants. Against this background, Noesis's candidate contribution is narrow: the admission policy, not memory beyond a context window in general. Benchmarks such as LoCoMo [16] and LongMemEval [24] offer standard long-term-memory evaluations that this paper does not yet use; LongMemEval-V2 [25] (in progress) adds workflow memory.

Hindsight. Hindsight [13], the closest system, separates world facts, the agent's experiences, opinions, and entity observations, recalls through four channels under a token budget, and gives each opinion an evolving confidence, so typed memory, budgeted hybrid retrieval, and evolving confidence are not Noesis contributions. The differences are narrower: Hindsight's confidence belongs to opinions the *agent* forms and, in the version I examined, does not gate recall, whereas Noesis gates the *user's* own statements; Noesis runs on SQLite with a CPU embedding model and pools several AI tools into one store. I have not compared the two empirically.

Learned and graph-structured memory control. Surveys organize agent memory by forms, functions, and dynamics [9], and graph memory by extraction, storage, retrieval, and evolution [27]. GraphRAG [5] summarizes entity communities for global questions over a corpus, a different task from tracking one user's preferences; AgeMem [29] learns when to store, retrieve, update, sum-

marize, or discard; and MRAgent [10] explores an associative memory graph guided by reasoning. Noesis instead uses fixed rules; its intended advantages (transparency, low overhead, portability) are unmeasured here apart from the latencies in §3.1, and task performance is not compared.

Memory security. MINJA [4] plants malicious memories through ordinary queries, and a survey [15] argues that memory security spans writing, storage, retrieval, execution, sharing, and forgetting, and cannot be added at retrieval time alone. Noesis's gate acts at retrieval and judges how firmly something was said, so a firmly worded planted statement passes (§9), and a shared store carries a record written in one tool into the others.

Retrieval-augmented generation. RAG [14] and successors retrieve per-query context. Self-RAG [2] and FLARE [11] improve retrieval quality or adaptivity, but address corpus QA, not a user's evolving personal memory. My scale experiment (§5.3) shows why naive inject-everything retrieval fails for personal memory: cost grows linearly and ranking collapses. Noesis pairs hybrid retrieval (BM25 + vector + RRF, with time-decay weighting) with a hard token budget and a confidence gate.

Personalization. PALR [28] and related work personalize outputs from user interaction history, typically for recommendation-style tasks. Chat-tool vendor memory features (e.g., ChatGPT memory) are per-product and closed. Noesis's cross-tool experiment (§5.4) demonstrates a structural advantage of user-scoped rather than tool-scoped memory.

Visualization of embedding spaces. t-SNE [23] and UMAP [17] project high-dimensional embeddings for inspection; force-directed layouts [6] model attraction/repulsion in 2D. My star map (§6) extends force-directed layout to 3D and adds weak axis springs, so semantic clusters still form freely while time reads left to right and confidence bottom to top.

3 System Design

Noesis sits between AI tools and model providers (Figure 1). API clients point at a local OpenAI-compatible proxy, which injects memory into the system prompt, forwards the request to the configured provider (cloud or local), and captures the conversation for memory formation; Claude Desktop connects through an MCP server and the browser extension through a WebSocket.

3.1 Thought taxonomy and two-phase capture

Conversational memory is typed into seven categories with distinct persistence semantics (Table 1). An *identity* thought ("I am a backend engineer") should outlive an *event* ("I tried Bun yesterday"), and the taxonomy encodes this as type-specific half-lives.

Capture is two-phase. *Phase 1* is synchronous: embed, deduplicate by hash, insert as *tentative* (median 21 ms, p99 684 ms over 100 adds on an x86-64 Mac). *Phase 2* runs in the background, so the conversation never waits for it; building the injected context adds a median of 21 ms (p99 55 ms) per request. In *Phase 2*, an extractor (a small cloud LLM, or a deterministic fallback) classifies the thought; a confidence scorer updates its status; a supersession check retires contradicted predecessors; the cold store (Markdown + YAML frontmatter in an Obsidian vault) is updated for human readability and editability.

Table 1: Thought taxonomy with type-specific half-lives and injection priority.

Type	Definition	Half-life (days)	Priority
identity	Who the user is	∞	highest (always)
preference	Explicit preference	90	high
position	Stance on a topic	90	high
question	Open question under exploration	30	medium
synthesis	Insight from merged ideas	180	medium
event	Happened / decided	7	low (recent)
contradiction	Self-contradictory judgment	∞	flagged

3.2 Confidence lifecycle

Every thought begins *tentative*: stored, but never injected and invisible to retrieval—the "capture threshold low, injection threshold high" principle. Four signals promote a thought: *repetition* (the same stance recurs across sessions), *assertion strength* ("I've decided" outranks "maybe"), *cross-tool consistency* (distinct phrasings of a stance from different tools), and *absence of contradiction*. The scorer combines them as $0.35\times$repetition$+0.25\times$assertion$+0.25\times$no-contradiction$+0.15\times$cross-tool, each scored from 0 to 1; a thought becomes provisional at 0.45 and settled at 0.75. A single firmly stated thought with no conflict ("I prefer X", assertion 0.8) therefore scores exactly 0.45 and becomes injectable, while settling always requires repetition: at least three similar earlier thoughts with cross-tool corroboration, or six without. *Provisional* thoughts inject when topic-relevant; *settled* thoughts always inject. The result is a rule-based admission score: it decides what may be injected, not whether a thought is true, correctly attributed, still valid, or cleared for reuse, and a score of 0.75 is not a calibrated 75% probability of being correct [8]. Confidence evolves incrementally: reinforcement $+0.10\,(1-C)$, conflict -0.20, and time decay $C \leftarrow C\,e^{-0.693\,\Delta t/t_{1/2}}$ with $t_{1/2}$ set by type (Table 1).

3.3 Cluster-aware supersession

When a user changes their mind ("I moved from MySQL to PostgreSQL"), the old stance should retire. Embedding similarity is the wrong signal here: contradictory stances mention different entities and therefore have *low* similarity. Noesis detects mind-changes by requiring jointly (1) a replacement verb ("switched to", "migrated to", …, EN+ZH), (2) the same domain via a keyword map (both stances are about *databases*), (3) stance-like type, and (4) injectable status of the new thought. Matched predecessors are marked *superseded*—excluded by every retrieval path—and linked via `superseded_by`. In my evaluation (the same 30 scenarios, run once before and once after the fix) this reduced stale-stance leakage from 93% to 7% (Table 2).

Table 2: Supersession fix: stale-stance leakage before/after (n=30 scenarios).

Metric	Before	After	Δ
Old stance leaked into context	28/30 (93%)	2/30 (7%)	−87pp
New stance surfaced	12/30 (40%)	28/30 (93%)	+53pp
Ideal (new shown, old retired)	1/30 (3%)	27/30 (90%)	+87pp

3.4 Selective retrieval and injection

At query time (Figure 2), the semantic channel retrieves up to eight thoughts by dense similarity; once a user has 300 thoughts it also runs BM25 and fuses the two rankings by reciprocal-rank fusion with time-decay weighting. Up to three recent and three core (identity and preference) thoughts are then appended, duplicates are dropped, and a confidence gate excludes everything below *provisional*. A hard token budget (1,200 tokens) truncates the remainder, and the result is injected into the system prompt as compact structured lines. A one-hop expansion over vault links currently adds nothing (§7).

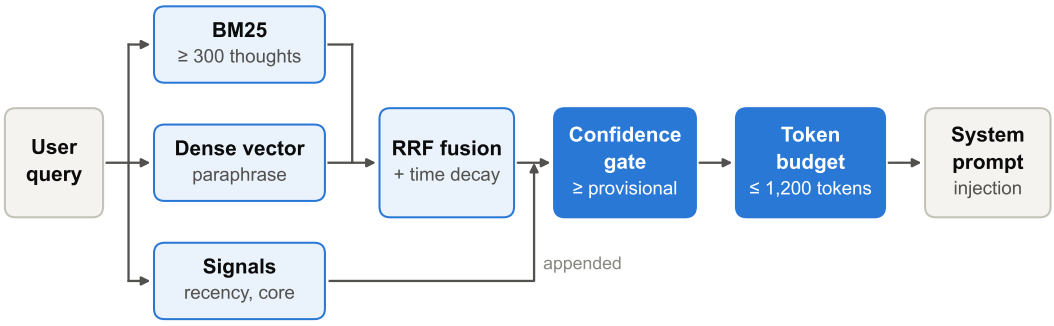


Figure 2: Retrieval and injection pipeline. Dense and BM25 rankings fuse via RRF (BM25 from 300 thoughts); recency and core-fact thoughts are appended; a confidence gate removes non-injectable thoughts; a token budget caps injected context.

4 Evaluation Setup

I evaluate four research questions:

- **RQ1**: Does memory injection raise the keyword-hit rate on personal questions? (A/B, end-to-end, §5.1)
- **RQ2**: Which components contribute? (ablation, retrieval-layer, §5.2)
- **RQ3**: How do cost and ranking scale with memory size? (§5.3)
- **RQ4**: Does cross-tool sharing work? (§5.4)

Models. Gemini Flash-Lite (cloud; `gemini-flash-lite-latest`, queried 30 June 2026), gemma3:4b and qwen2.5:3b (local via Ollama), spanning capability tiers. Answers used a 150-token limit and default sampling (no temperature set), one per question and condition; the Gemini alias and Ollama tags are not immutable, and local model digests were not recorded.

A/B design. 30 synthetic English user profiles, each with 5 build statements (an identity statement plus preferences, positions, and decisions) followed by 5 test questions whose expected keyword is known from the build statements. Each build statement is stated once and firmly ("I prefer…", "I decided…"), which is enough to reach provisional (§3.2). For each question the model answers twice: *with memory* (via the Noesis proxy, which injects context) and *without memory* (direct to

the provider). All models are evaluated on the identical first 110 questions (22 profiles). An answer counts as a hit if it contains the expected keyword, and I report the keyword-hit rate; paired outcomes are tested with McNemar's χ^2.

Retrieval-layer experiments. Ablation, scale, and cross-tool experiments run against the real pipeline without LLM calls, measuring Recall@5, MRR, the recall and size of the full injected context, tentative leakage, and injected tokens. They call the Python API directly, and the cross-tool experiment simulates tools by labeling each statement's source.

Integration routes. Only the proxy, which injects context and stores each exchange automatically, is evaluated end to end, in the A/B and end-to-end runs (§5.1). MCP tools are model-controlled [19]: nothing is stored or injected unless the model calls `remember` or `recall`, so exposing them does not ensure timely use, and the MCP server has unit tests only. The browser extension only captures turns from chat web apps and is not evaluated.

Reproducibility. Every experimental number in this paper is recomputed from the raw result files by a released script (`experiments/verify_paper.py`), which also reads the lifecycle constants from the source. The ablation and scale experiments were re-run in October 2026 with the June software versions (Python 3.12.11, torch 2.2.2, transformers 4.44.2, sentence-transformers 3.0.1, all-MiniLM-L6-v2); they reproduced the June numbers exactly before the corrected leak metric and the top-5 baseline were added. The ablation was later corrected further (§9). `verify_paper.py` is an arithmetic audit: it checks that each reported number follows from the saved per-question records, not that each answer was scored correctly. Those records are incomplete. The local-model A/B runs kept the first 150 characters of each answer and the end-to-end run the first 100, although hits were scored on the full answer; the Gemini A/B run kept no answer text, because its records were recovered from the run's console log, which printed only hit marks; and no run kept the full prompt or the injected context. The released files therefore support checking the arithmetic, not re-scoring the answers. The code, data, and paper sources used here are tagged `v1.0-preprint` in the repository.

5 Results

5.1 RQ1: Memory injection raises the keyword-hit rate across model tiers

Table 3: A/B keyword-hit rates on identical n=110 questions. All $p < 0.001$ (McNemar, df=1).

Model	n	**With**	**Without**	Δpp	χ^2
Gemini Flash-Lite	110	85.5%	41.8%	**+43.6**	42.48
gemma3:4b	110	82.7%	34.5%	**+48.2**	51.02
qwen2.5:3b	110	62.7%	37.3%	**+25.5**	16.57

Table 3 and Figure 3 show the headline result. Three findings stand out. First, *injection helps everywhere*: all three models improve significantly, and memory *hurt* in only 10 of 330 paired answers (3.0%). Second, *memory levels the field between local and cloud*: with injection, gemma3:4b (4B parameters, local) reaches 82.7%, nearly matching Gemini Flash-Lite's 85.5%, while its no-memory baseline (34.5%) trails Gemini's 41.8%. Third, *the smallest model gains least*: qwen2.5:3b goes

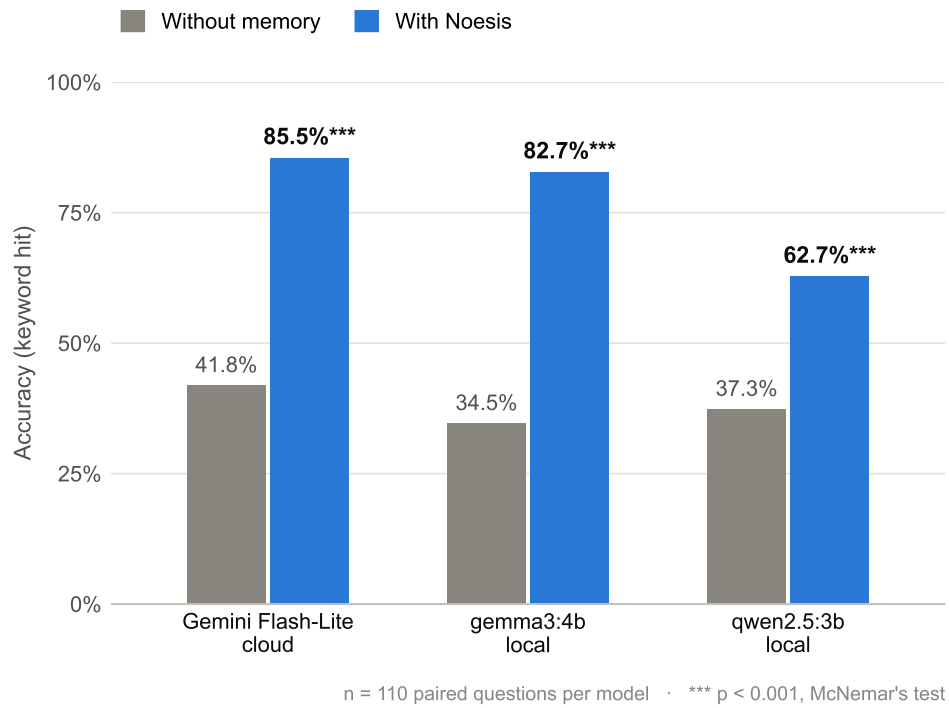


Figure 3: A/B keyword-hit rate across models (n=110 each). All improvements significant at $p < 0.001$.

through the same retrieval pipeline but reaches only 62.7%. A model-capability ceiling is one possible reason, but the saved runs do not record the injected context, so it remains a hypothesis.

The hit rate alone does not separate retrieval strategies at this scale. In a separate end-to-end run on 50 questions (10 profiles, Gemini Flash-Lite), Noesis, injecting every stored memory, and injecting the five most recent memories scored 41, 40, and 41 of 50, against 22 of 50 with no memory. With only 3–9 stored memories per profile in that run, all three strategies inject nearly the same context; they diverge on filtering and cost as the store grows (§5.2–§5.3). The A/B result therefore shows that personal context helps on these tasks, not that the lifecycle beats simpler ways of supplying the same memories.

5.2 RQ2: Semantic retrieval drives recall; gating keeps tentative thoughts out

Each Noesis variant runs the real context builder with exactly one component switched off, so signal order, deduplication, and the token budget stay the same; the baselines draw from every stored memory, tentative ones included. Status and confidence are assigned when the test store is built, so this suite tests whether retrieval respects those labels, not whether the lifecycle derives them from conversation. With 28 memories the semantic channel is dense retrieval alone (BM25 fusion starts at 300 memories). Stale-memory leakage is measured only by the supersession test (Table 2).

Removing semantic retrieval collapses recall (99%→24%), confirming it as the main source of recall (Table 4). Removing the confidence gate leaves recall unchanged but lets tentative thoughts into 73 of the 573 injected slots, where the full system injects none—gating is what keeps *tentative* material out of the prompt. Ungated top-5 retrieval ranks as well as Noesis here (Recall@5 99%, MRR 1.000), as it does at scale (§5.3), but injects a tentative thought in 19 of its 250 slots. A separate poisoning test gives the same picture: with 5 settled facts and 8 hedged statements that

Table 4: Ablation and baselines (50 queries over 28 memories, 5 of them tentative; retrieval-layer, 2,000-token budget). A retrieved memory counts only if it is the ground-truth memory itself. Recall@5 and MRR use the first five memories in injection order, and a query that returns fewer than five memories is scored on what it returns; context recall, items, and leak cover everything the configuration injects, and leak counts tentative memories among those slots.

Config	Recall@5	MRR	Context recall	Items	Leak
Noesis (full)	**99%**	**1.000**	**100%**	**10.9**	**0/546**
w/o core-fact signal	99%	1.000	100%	9.7	0/487
w/o recency signal	99%	1.000	100%	9.7	0/484
w/o confidence gating	99%	1.000	100%	11.5	**73/573**
w/o semantic retrieval	24%	0.105	24%	5.0	0/250
Top-5 RAG	99%	1.000	99%	5.0	19/250
Recent-window	17%	0.084	17%	5.0	50/250
Random	12%	0.063	12%	5.0	51/250

contradict them, ungated top-5 retrieval returned a hedged statement in 13 of 25 slots across 5 queries, and Noesis in none. Both tests use hedged statements; a firmly stated false one would pass the gate (§9). Removing the core-fact or recency signal changes nothing measurable: each adds about one memory per query (10.9 vs. 9.7 items) without changing recall. The builder adds these signals after the eight semantic results, so they never reach the first five slots; whether they help on queries the semantic channel misses needs a dedicated query set.

5.3 RQ3: Retrieval keeps cost flat as memory grows

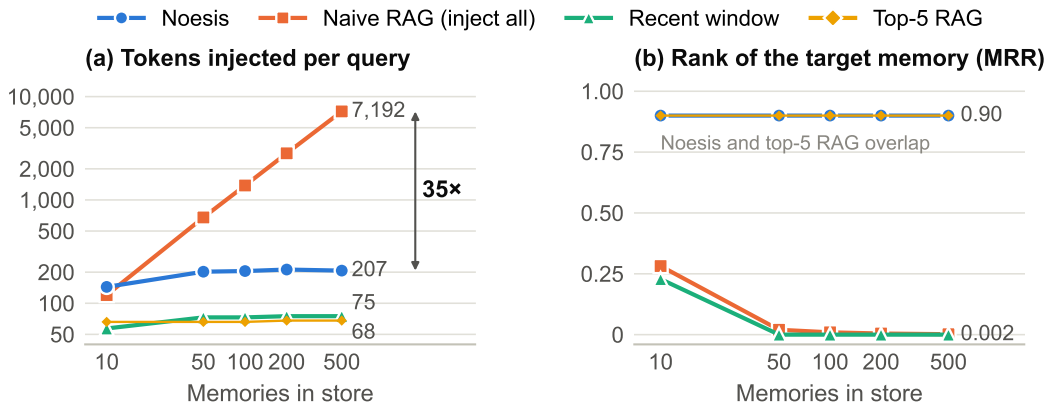


Figure 4: Scale behavior from 10 to 500 stored memories. (a) Noesis's injected tokens stay flat (∼207) while naive RAG (inject all) grows to 7,192 (35 times); top-5 RAG stays at 66–68. (b) Noesis and top-5 RAG hold MRR at 0.90; naive RAG collapses to 0.002 as targets are buried; recent-window loses recall entirely past n=10.

Figure 4 sweeps memory size from 10 to 500 with 10 known targets embedded among fillers. Noesis's injected cost is flat (144→207 tokens) because retrieval returns a small, confidence-gated set; the 1,200-token budget is never reached. MRR holds at 0.90: 9 of the 10 targets sit at rank 1,

and the tenth is never retrieved. Naive RAG keeps recall only by injecting everything: 7,192 tokens at n=500, with the target buried near rank 500 (MRR 0.002). Recent-window recall collapses to 0% beyond n=10: the window forgets everything older. A standard top-5 RAG, which embeds the query and injects its five nearest memories, does as well as Noesis on this benchmark: the same 9 of 10 targets at rank 1, with 66–68 tokens per query. Flat cost and good ranking therefore come from retrieval itself rather than from anything specific to Noesis; what Noesis adds is the gate (§5.2), which top-5 RAG lacks. The methodological takeaway: *recall alone is misleading*; rank and token cost must be reported together.

5.4 RQ4: Cross-tool sharing vs. structural isolation

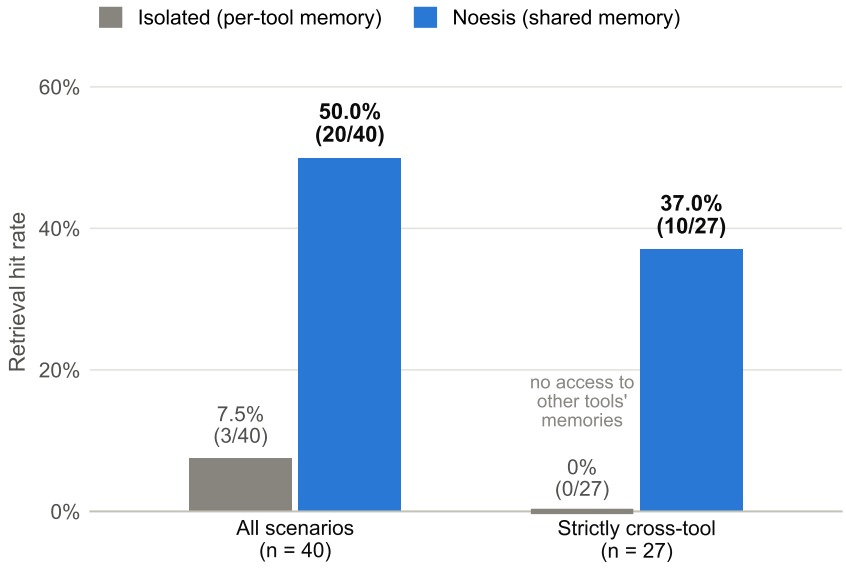


Figure 5: Cross-tool retrieval. Isolated per-tool memory scores 0% on strictly cross-tool queries by construction.

Of 40 scenarios, 27 state a stance in one tool and ask about it from another; in 7, the asking tool was one of two tools that stated it; and in 6, a third tool asks after two others stated it (Figure 5). On the 27 strictly cross-tool queries, Noesis retrieves 37% (10/27) and the isolated baseline 0%—not because it ranks poorly, but because the memory is not in its partition, so that 0% holds by construction. The 7 scenarios where the asking tool did see the stance are a fairer comparison: isolated memory retrieves 3 of 7 and Noesis 6 of 7. Over all 40 scenarios, Noesis retrieves 50% and the isolated baseline 7.5%. This is the structural argument: per-tool memory systems cannot share by design, and no amount of per-tool tuning closes the gap. The comparison therefore shows the value of access, not an advantage over another shared-memory system given the same records. Noesis's misses trace to retrieval precision (the memory exists but ranks below top-k), not to sharing.

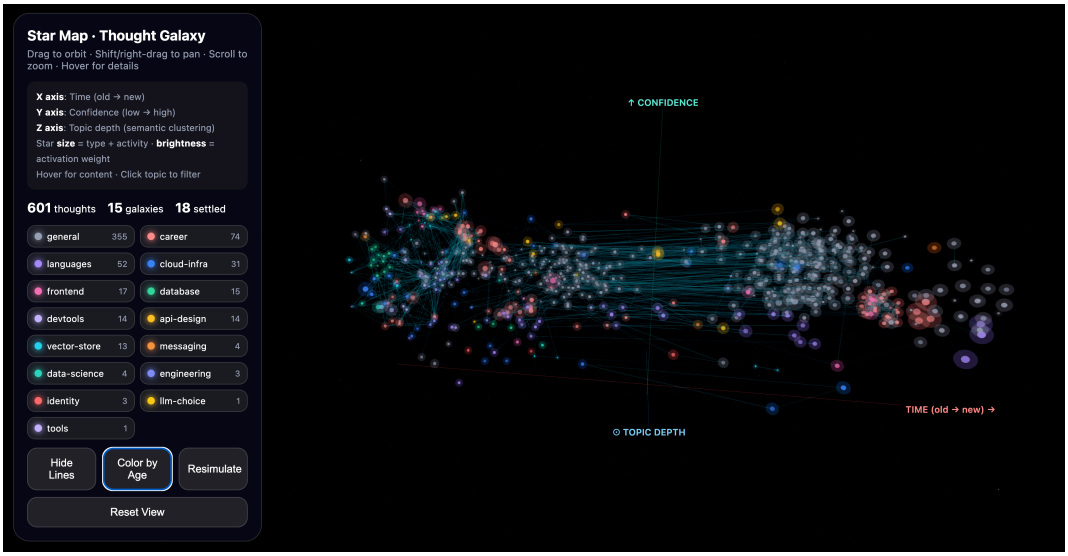


Figure 6: The star map. Semantic neighbors attract into volumetric constellations. Time runs from older (left) to newer (right), height reflects confidence, and color encodes topic; star size encodes type and activation weight, and brightness also tracks activation weight.

6 The Star Map Visualization

The star map displays the stored thoughts as a galaxy (Figure 6). Each thought is a star placed by a short force simulation. Weak springs pull each star toward its creation time on the horizontal axis and its confidence on the vertical axis; *semantic attraction* pulls it toward its most similar neighbors (embedding cosine ≥ 0.45); and *repulsion* keeps stars from overlapping while pushing different topics apart. The springs are weak enough that attraction wins locally, so clusters form from embedding similarity while time and confidence stay readable at a glance; topic labels only decide which clusters repel. This makes the memory store inspectable rather than a black box: older thoughts sit to the left, settled ones sit high, and hovering a star shows its text and confidence. The store shown is the local development store, populated by the experiment and demo runs rather than by real use: of its 601 thoughts, 355 sit in the catch-all "general" topic, which the extractor assigns when no known topic matches, and 18 have settled. Even so, the view surfaces such patterns at a glance; here it shows how coarse topic assignment still is. The viewer is a local web tool shipped with the system; it opens the store read-only, so inspecting memory never changes it. The map is a display only: its edges are computed by the viewer and never affect retrieval, so it says nothing about whether graph structure improves answers.

7 Discussion

When memory helps most. Gains concentrate on questions that are unanswerable without personal context (job, stack, stances). On mainstream questions both groups often answer correctly, which compresses the measured gap on those items. Other factors push the other way (see Limitations), so these results should not be read as a bound in either direction.

Memory can close the local–cloud gap. gemma3:4b posts the largest gain (+48.2pp; +139% over its baseline), approaching Gemini Flash-Lite once memory is injected. The benefit is not

monotonic in model size: qwen2.5:3b gains least (+25.5pp; §5.1). Memory helps small local models most when they can use the injected context, which matters for privacy-sensitive and offline settings.

Retrieval precision is the current bottleneck. The cross-tool misses and several A/B misses reduce to ranking failures, not architecture failures: the memory is stored but ranks below the cut. Improving ranking (rerankers, query expansion) is the highest-leverage next step. The supersession fix's residual 7% is different: in both remaining leaks the detector never fired, once because "I now believe microservices are better" uses no listed replacement verb and names no known domain, and once because "I switched to Node.js" has the verb but Node.js is missing from the domain map. A broader mind-change detector is the matching fix.

Noesis inside an agent harness. An execution harness manages a task's control flow, tools, state, and recovery [1]; Noesis would sit inside one as a memory service, not as orchestration. None of this is implemented or evaluated here. Before a model step, the harness would insert approved memories for the current user, project, and access scope as a bounded, sourced block, and afterward send candidate memories with their sources. LangGraph separates thread-scoped checkpoints from a cross-thread store [12]; Noesis could fill only the second role. An integration would also need corrections, deletion, audit, and source lineage. Three gaps stand out: repetition counts similar records whatever their source, and the proxy stores each request's full history, so replays and assistant paraphrases can inflate confidence (unmeasured); new memories become injectable only after asynchronous scoring; and injected memories are neither marked as data nor deduplicated against the harness's own summaries.

Graph structure. The evaluated system is not a graph-based reasoner: supersession links are implemented and evaluated (Table 2); topic clusters, which limit which records the scorer compares, and wiki links are not evaluated separately; and the one-hop expansion over vault links cannot add a memory (it writes 8-character identifiers but looks up 12-character ones), while the ablation and scale stores have no links. Every reported result therefore comes from flat retrieval.

Negative results. I report that the supersession check initially never ran, so stale stances leaked 93% of the time until it was fixed (§3.3); that core-fact and recency signals added no recall in my ablation suite; that a standard top-5 RAG matches Noesis's ranking at lower cost on the scale benchmark; that the 3B model gains least; and that the graph expansion never adds a memory. I consider these findings part of the contribution.

8 Future Work

Thought branching. When a new judgment deviates from history by more than a threshold ($\delta = 1 - \cos > 0.45$), Noesis should create a *branch* linked via `evolved_from` rather than superseding, preserving the full trajectory of a stance. This enables metrics I cannot compute today: stabilization depth, branching rate, and post-branch divergence—a longitudinal record of how a user's positions evolve.

Structured injection. Replacing flat context lines with a structured `user_thought_graph` (anchor topic, trajectory with weights, cross-links) may help small models use injected memory.

Stronger evaluation. LLM-as-judge scoring to complement keyword hits; distractor questions; a second cloud model to replicate the Gemini result; longitudinal deployment data beyond synthetic profiles. Some questions need tentative thoughts ("what alternatives was I considering last week?"), which the gate withholds; tests should compare excluding them with retrieving them alongside their uncertainty and source, adapting the risk–coverage view of selective prediction [7].

Harness-level evaluation. With model, tasks, tools, and budget fixed, compare native harness memory, a simple shared store, and Noesis on task completion, preference use, resumption, stale information, project isolation, and needless retrieval, counting cost from capture to generation. Counting repetition only across distinct user statements would close the first gap in §7.

Typed temporal claim graph. Claims linked to user, project, source, tool, and time, with support, contradiction, derivation, and supersession relations, would let "PostgreSQL for analytics" and "MySQL for a legacy app" coexist; Zep [22] is the closest precedent. Derived claims are not independent observations, so graph centrality should not raise confidence. The deciding experiment crosses flat and graph-backed stores with and without the same gate, all else fixed.

9 Limitations

Scoring is keyword-hit, which is objective but coarse: it checks whether an answer contains the expected keyword, not whether the answer is right. "You do not prefer Python" contains the same keyword as "you prefer Python", and a correct answer worded differently or given in another language counts as a miss. Personas are synthetic; build/test alignment may overstate real-world gains, and real conversations are noisier. The cloud A/B uses one provider (Gemini Flash-Lite) and n=110 rather than the planned 150: free-tier rate limits stopped the 30 June run after 111 questions. A later session added 35 more, but because the `gemini-flash-lite-latest` alias may have pointed to a different model version by then, I use only the first 110 questions (22 complete profiles) from the single June run; all significance thresholds are still exceeded by wide margins. Retrieval-layer experiments measure what reaches the prompt, not final answer quality. The supersession detector relies on replacement verbs and a domain keyword map; paraphrased mind-changes outside its lexicon will be missed. The supersession fix was developed and validated on the same 30 scenarios, so its 93% → 7% result is in-sample; held-out, paraphrased mind-changes, unseen domains, and legitimate simultaneous preferences, which a detector that matches by domain could wrongly retire, are needed to measure it fairly. The scale experiment's naive-RAG baseline injects every stored memory; the top-5 baseline added in §5.3 is the stronger comparison, and on this benchmark it ties Noesis on ranking. The ablation and poisoning leak counts were corrected in October 2026: the original keyword-based metric also counted settled memories that merely shared a word with a tentative one (for example, "Kafka"). After an external review, the ablation now scores exact memory identity (phrase matching had credited any memory sharing wording such as "I prefer"), drops "Precision@5" (a non-tentative rate), runs every variant through the same context builder (the old core-fact and recency rows could not differ from the full system), applies the memories' ages (previously dropped), and lets the recent-window and random baselines return tentative memories. This lowered the weak baselines (random 45% → 12%, w/o semantic 49% → 24% Recall@5); Noesis moved from 100% to 99%. All results come from my own runs and have not been independently replicated.

Noesis also adds an attack surface. The local proxy sees every request, including provider API keys and full conversations; enabling cloud extraction sends conversation text to a third-party API,

which weakens the local-first guarantee; and because a single firmly stated sentence already reaches provisional (§3.2), a planted or mistaken statement can enter future prompts. The confidence gate filters hedged statements but is not a defense against deliberate poisoning.

10 Conclusion

Injecting a user's own memories raised the keyword-hit rate of all three models on personal questions; whether Noesis's lifecycle beats simpler ways of supplying the same memories is not yet shown. Its confidence gate kept tentative and hedged statements out of the prompt in constructed tests where plain retrieval let them in, though it judges how firmly something was said, not whether it is true. Retrieval keeps cost flat as memory grows (35 times cheaper than injecting everything at n=500, though a standard top-5 RAG is cheaper still), and a shared per-user store gives every connected tool the same records, which per-tool stores cannot. The result is a local-first layer that brings a 4B local model close to a small cloud model's keyword-hit rate (82.7% vs. 85.5% for Gemini Flash-Lite; not significantly different on this set, which does not make them equivalent)—and that keeps the user's reasoning in the user's own vault.

Acknowledgments

AI coding assistants were used in this work. ZCode (Z.ai, GLM models) assisted with implementing the experiment scripts and drafting the original experiment reports. Claude (Anthropic) was used to review the manuscript, revise its text, redesign the figures, write the verification script (`experiments/verify_paper.py`), and re-run the retrieval-layer experiments in October 2026. All reported numbers are checked against the raw results by that script, and the author takes full responsibility for the paper's content.

References

[1] Anthropic. Effective harnesses for long-running agents. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents, 2025. Engineering blog, November 26, 2025.

[2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11511.

[3] Prateek Chhikara, Dev Khant, Sparsh Aryan, et al. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. ECAI 2025.

[4] Shen Dong, Shaochen Xu, Pengfei He, et al. Memory injection attacks on LLM agents via query-only interaction. *arXiv preprint arXiv:2503.03704*, 2025.

[5] Darren Edge, Ha Trinh, Newman Cheng, et al. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[6] Thomas M. J. Fruchterman and Edward M. Reingold. Graph drawing by force-directed placement. *Software: Practice and Experience*, 21(11):1129–1164, 1991.

[7] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[8] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

[9] Yuyang Hu, Shichun Liu, Yanwei Yue, et al. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*, 2025.

[10] Shuo Ji, Yibo Li, and Bryan Hooi. Memory is reconstructed, not retrieved: Graph memory for LLM agents. *arXiv preprint arXiv:2606.06036*, 2026. ICML 2026.

[11] Zheng Jiang, Frank F. Xu, Luyu Gao, et al. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7969–7992, 2023.

[12] LangChain. Persistence. https://docs.langchain.com/oss/python/langgraph/persistence, 2026. LangGraph documentation, accessed October 17, 2026.

[13] Christopher Latimer, Nicolò Boschi, Andrew Neeser,

et al. Hindsight: Structured agent memory that retains, recalls, and reflects. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 275–285, 2026. doi: 10.18653/v1/2026.acl-demo.27.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[15] Zehao Lin, Xixuan Hao, Renyu Fu, et al. A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle. *arXiv preprint arXiv:2604.16548*, 2026. EMNLP 2026.

[16] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[17] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[18] Mem0. Introducing OpenMemory MCP. https://mem0.ai/blog/introducing-openmemory-mcp, 2025. Vendor announcement, Mem0 blog, May 13, 2025.

[19] Model Context Protocol. Model context protocol specification, revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28/architecture, 2026. Accessed October 17, 2026.

[20] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[21] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[22] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[23] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.

[24] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025.

[25] Di Wu, Zixiang Ji, Asmi Kawatkar, et al. LongMemEval-V2: Evaluating long-term agent memory toward experienced colleagues. *arXiv preprint arXiv:2605.12493*, 2026. Work in progress.

[26] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[27] Chang Yang, Chuang Zhou, Yilin Xiao, et al. Graph-based agent memory: Taxonomy, techniques, and applications. *arXiv preprint arXiv:2602.05665*, 2026.

[28] Liangwei Yang, Zhiwei Chen, Lixin Xia, Chen Yang, Yu Zhang, and Ben Kao. PALR: Personalization aware LLMs for recommendation. *arXiv preprint arXiv:2305.07622*, 2023.

[29] Yi Yu, Liuyi Yao, Yuexiang Xie, et al. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*, 2026. ACL 2026.

[30] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.